

TONIGHT

Working with tables: pandas + SQL

Five verbs for one table. Four ways to combine two. Then the same verbs in SQL, the language company databases speak.

Data Analysis & Machine Learning for Business

Andrey Shumilin

Short recap.

ONE PICKING TASK

StartTask the picker accepts the task and takes an empty container



ScanCell walk to the cell the terminal shows, scan the shelf barcode – around 30 cells per task (5 to 50)



DropBoxing container full: leave it at a drop point, take an empty one – 1 to 4 containers per task, each with its own id



FinishTask every line done, the task closes

One line event = **one row** of the table you loaded last week.

NOTE

At first → **Basis** (excalidraw)

INSIDE ONE SCANCELL

Each scanned cell ends in one of four line events – and each of those is **one row** in our table.

- **PickCorrectItem** scan the item: right one, goes into the container
- **PickWrongItem** scan the item: wrong one, keep searching this cell
- **SkipItem** the item is not in the cell at all
- **MarkDefectItem** the item is there but damaged

Watching me type teaches almost nothing.

01

I can run the code and narrate what happens, but that is dull for you and for me. By the third cell your attention is somewhere else and the thread is lost.

So my part

Give you the general idea, point your attention at the few things that actually matter, and show the starting points for each direction.

02

You understand a tool only by using it yourself. Do it often and you build **an eye for data**: a few statistics are enough to form a hypothesis and to tell a promising path from a dead end.

→ Your part

Think a lot. Write code for a task of your own. And find someone experienced to give you feedback on what is right and what is wrong.

Tonight is the idea and the pointers. The skill is built after class.

Tonight, in four steps.

01

Five verbs for one table

select, filter, sort, derive, aggregate — your Excel clicks, written down

02

groupby

split → apply → combine, and the row that quietly disappears

03

Joining two tables

one key, four join types, and what happens to rows without a partner

04

Databases and SQL

the same five verbs in the language company databases speak

**What do you usually do
in **Excel** to analyse data?**

A table answers with five verbs.

You already do all five in Excel, with the mouse. Tonight you learn how to write them down.

01

select

in Excel: hide columns
`df[["a", "b"]]`

02

filter

in Excel: dropdown
filter
`df[mask]`

03

sort

in Excel: sort A→Z
`.sort_values()`

04

derive

in Excel: a formula
column
`df["c"] = ...`

05

aggregate

in Excel: the pivot
table
`.groupby(...)`

Verbs 1–4 you recognize. Verb 5 is **tonight's real work.**

Demo 01

The five verbs on the warehouse, one at a time, with a prediction before every cell.

01 Select: one bracket a column, two a smaller table

02 Filter: the mask alone first, a column of True/False

03 Sort: the fullest cell ever scanned

04 Derive: date, hour, and a night flag with | & ~

05 Aggregate: groupby, learned on eight rows before the warehouse

06 The detective: where the holes are, and why

Predict the answer out loud, then run the cell.

groupby = split → apply → combine

ASK What aggregation functions do you know?

THE TABLE, 8 ROWS

	worker	action	seconds
0	Anna	pick	5
1	Boris	pick	7
2	Anna	pick	6
3	Clara	scan	2
4	Boris	pick	9
5	Anna	scan	3
6	None	pick	4
7	Clara	pick	8



Count the rows in each pile. That is the **whole idea**.

Eight rows in, seven rows counted.

The row with the empty key joined no pile, so the three piles add up to seven.

```
.groupby("worker").size()
```

worker	size
Anna	3
Boris	2
Clara	2
<i>None</i>	±

$$3 + 2 + 2 = 7 \neq 8$$



```
.groupby("worker", dropna=False).size()
```

worker	size
Anna	3
Boris	2
Clara	2
<i>NaN</i>	1

$$3 + 2 + 2 + 1 = 8 \checkmark$$

groupby leaves out rows whose key is empty. No error, no warning: the cell runs green and the total is short. On our warehouse that is a fifth of all wrong picks missing from “errors by item”.

THE HABIT

Not sure how a function behaves? **Build a five-row table where you know the answer, and look.**

The answer is in two tables at once.

toy_picks, 6 rows

worker	item	cell_id	seconds
Anna	mug	A	5
Anna	kettle	A	7
Boris	mug	B	6
Clara	lamp	C	2
Boris	mug	D	9
Clara	cable	<i>None</i>	3

toy_cells, 4 rows

cell_id	zone
A	Z1
B	Z1
C	Z2
E	Z3

How can we calculate in which zone Anna has picked a mug?

How can we join these two tables? What is the simplest way?

The key is `cell_id` — the column both tables mean the same thing by.

Inner keeps only the matches.

toy_picks, 6 rows

worker	item	cell_id	seconds
Anna	mug	A	5
Anna	kettle	A	7
Boris	mug	B	6
Clara	lamp	C	2
Boris	mug	D	9
Clara	cable	None	3

toy_cells, 4 rows

cell_id	zone
A	Z1
B	Z1
C	Z2
E	Z3

Cell **D** was picked but is missing from the reference. Clara's last pick has an empty key. Cell **E** exists but was never picked.

inner keeps a row only if its key was found in both tables.

☒ key matched

☒ kept, gap filled with NaN

☐ dropped, no warning

HOW="INNER", THE DEFAULT

worker	item	cell_id	zone
Anna	mug	A	Z1
Anna	kettle	A	Z1
Boris	mug	B	Z1
Clara	lamp	C	Z2
Boris	mug	D	
Clara	cable	None	

4 rows, two picks gone

HOW="LEFT"

HOW="OUTER"

HOW="RIGHT"

This is the default. Two picks are gone and nothing warned you.

Left keeps all of mine.

toy_picks, 6 rows

worker	item	cell_id	seconds
Anna	mug	A	5
Anna	kettle	A	7
Boris	mug	B	6
Clara	lamp	C	2
Boris	mug	D	9
Clara	cable	None	3

toy_cells, 4 rows

cell_id	zone
A	Z1
B	Z1
C	Z2
E	Z3

Cell **D** was picked but is missing from the reference. Clara's last pick has an empty key. Cell **E** exists but was never picked.

left keeps every row of the left table and fills the gaps with NaN.

☒ key matched

☒ kept, gap filled with NaN

☐ dropped, no warning

HOW="INNER", THE DEFAULT

worker	item	cell_id	zone
Anna	mug	A	Z1
Anna	kettle	A	Z1
Boris	mug	B	Z1
Clara	lamp	C	Z2
Boris	mug	D	
Clara	cable	None	

4 rows, two picks gone

HOW="LEFT", LIKE VLOOKUP

worker	item	cell_id	zone
Anna	mug	A	Z1
Anna	kettle	A	Z1
Boris	mug	B	Z1
Clara	lamp	C	Z2
Boris	mug	D	NaN
Clara	cable	None	NaN

6 rows, honest gaps

HOW="OUTER"

HOW="RIGHT"

left is the analyst's default: all of my rows, honest gaps where the reference says nothing.

Outer keeps everything.

toy_picks, 6 rows

worker	item	cell_id	seconds
Anna	mug	A	5
Anna	kettle	A	7
Boris	mug	B	6
Clara	lamp	C	2
Boris	mug	D	9
Clara	cable	None	3

toy_cells, 4 rows

cell_id	zone
A	Z1
B	Z1
C	Z2
E	Z3

Cell **D** was picked but is missing from the reference. Clara's last pick has an empty key. Cell **E** exists but was never picked.

outer keeps every row of both tables, so cell E shows up too.

☒ key matched

☒ kept, gap filled with NaN

☐ dropped, no warning

HOW="INNER", THE DEFAULT

worker	item	cell_id	zone
Anna	mug	A	Z1
Anna	kettle	A	Z1
Boris	mug	B	Z1
Clara	lamp	C	Z2
Boris	mug	D	
Clara	cable	None	

4 rows, two picks gone

HOW="LEFT", LIKE VLOOKUP

worker	item	cell_id	zone
Anna	mug	A	Z1
Anna	kettle	A	Z1
Boris	mug	B	Z1
Clara	lamp	C	Z2
Boris	mug	D	NaN
Clara	cable	None	NaN

6 rows, honest gaps

HOW="OUTER", KEEP EVERYTHING

worker	item	cell_id	zone
Anna	mug	A	Z1
Anna	kettle	A	Z1
Boris	mug	B	Z1
Clara	lamp	C	Z2
Boris	mug	D	NaN
NaN	NaN	E	Z3
Clara	cable	None	NaN

7 rows, every gap visible

HOW="RIGHT"

outer is the diagnostic: every gap on both sides becomes visible.

Right is the mirror.

toy_picks, 6 rows

worker	item	cell_id	seconds
Anna	mug	A	5
Anna	kettle	A	7
Boris	mug	B	6
Clara	lamp	C	2
Boris	mug	D	9
Clara	cable	None	3

toy_cells, 4 rows

cell_id	zone
A	Z1
B	Z1
C	Z2
E	Z3

Cell **D** was picked but is missing from the reference. Clara's last pick has an empty key. Cell **E** exists but was never picked.
right keeps every row of the right table instead.

☒ key matched

☒ kept, gap filled with NaN

☐ dropped, no warning

HOW="INNER", THE DEFAULT

worker	item	cell_id	zone
Anna	mug	A	Z1
Anna	kettle	A	Z1
Boris	mug	B	Z1
Clara	lamp	C	Z2
Boris	mug	D	
Clara	cable	None	

4 rows, two picks gone

HOW="LEFT", LIKE VLOOKUP

worker	item	cell_id	zone
Anna	mug	A	Z1
Anna	kettle	A	Z1
Boris	mug	B	Z1
Clara	lamp	C	Z2
Boris	mug	D	NaN
Clara	cable	None	NaN

6 rows, honest gaps

HOW="OUTER", KEEP EVERYTHING

worker	item	cell_id	zone
Anna	mug	A	Z1
Anna	kettle	A	Z1
Boris	mug	B	Z1
Clara	lamp	C	Z2
Boris	mug	D	NaN
NaN	NaN	E	Z3
Clara	cable	None	NaN

7 rows, every gap visible

HOW="RIGHT", RARELY USED

worker	item	cell_id	zone
Anna	mug	A	Z1
Anna	kettle	A	Z1
Boris	mug	B	Z1
Clara	lamp	C	Z2
NaN	NaN	E	Z3

5 rows, unpicked E appears

right is left seen from the other side. Rarely what you want, easy to reach for by mistake.

Two rules, and the Excel picture.

- ① The default is inner. Forget how= and unmatched rows are deleted without a word.
- ② An empty key never matches anything, not even another empty key. Our `item_id` is empty on 45% of rows, so an inner join throws away half the table.

COMING FROM EXCEL

A join is VLOOKUP, minus two guarantees. VLOOKUP returns the **first** match, a join returns **every** match. VLOOKUP only **adds columns**, a join can also **delete rows**.

- ✓ Counting catches two failures out of three: `assert len(after) == len(before)`. A wrong key leaves the count intact, and only you know what a column means.

What can go wrong?

Same two tables, one join, no error message. Name the ways the result can lie to you.

toy_picks, 6 rows

worker	item	cell_id	seconds
Anna	mug	A	5
Anna	kettle	A	7
Boris	mug	B	6
Clara	lamp	C	2
Boris	mug	D	9
Clara	cable	<i>None</i>	3

toy_cells, 4 rows

cell_id	zone
A	Z1
B	Z1
C	Z2
E	Z3

Three guesses, then we check them one by one.

Three ways a join goes wrong.

FAILURE 01

Lost rows

An inner join, plus keys that find no partner. The cell runs green and hands you a smaller table.

FAILURE 02

Multiplied rows

The key repeats on the other side, so the join returns every combination. Every sum after it is too big.

FAILURE 03

The wrong key

The join works, the count looks fine, and the answer means nothing. Only you know what the key means.

A join never errors. It hands you a table that looks fine, so **count your rows. That catches two failures out of three.**

```
assert len(after) == len(before)
```

A duplicated key multiplies rows.

toy_picks, 6 rows

worker	item	cell_id
Anna	mug	A
Anna	kettle	A
Boris	mug	B
Clara	lamp	C
Boris	mug	D
Clara	cable	None

toy_cell_items, cell A twice

cell_id	stored_item
A	mug
A	kettle
B	mug



merge(..., how="left")

worker	item	cell_id	stored_item
Anna	mug	A	mug
Anna	mug	A	kettle
Anna	kettle	A	mug
Anna	kettle	A	kettle
Boris	mug	B	mug
Clara	lamp	C	NaN
Boris	mug	D	NaN
Clara	cable	None	NaN

6 rows in → 8 rows out. Both of Anna's picks now exist twice.

Every sum and average after this join is wrong, and nothing complained. Insurance: `.is_unique` on the key before the join, assert after.

Demo 02

Joins on the warehouse, with all three failures reproduced on purpose.

- 01 The four join types, live on six rows
- 02 Meet the reference tables: cells, items, tags
- 03 Picks × topology: **the warehouse gets a geography**
- 04 Failure #1: the forgotten `how=`, and 139 thousand rows gone
- 05 Failure #2: a tag list makes the key non-unique
- 06 Failure #3: the right row count on the wrong key
- 07 Catalogue is not traffic: flip the join around

Before each join: will the row count go up, down, or stay?

You don't scroll a database. You ask it.

WHERE IT STARTS

Company systems

Terminals, tills and apps. Every action writes its row.



SQL LIVES HERE

Databases & the warehouse

Sizes no laptop holds, many people at once.



PANDAS LIVES HERE

Your laptop

The slice you asked for, refined and charted.

Pull with SQL, polish with pandas.

Tonight we run SQL on **sqlite**, a complete database that lives in a single file. No server, and it genuinely sits inside apps on your phone.

A query is five verbs in a fixed order.

SELECT user_id, COUNT(*) **AS** picks

select + aggregate

FROM picking_events

which table

WHERE event_type = 'PickWrongItem'

filter

GROUP BY user_id

aggregate: split into piles

ORDER BY picks **DESC**

sort

LIMIT 10

head()

SELECT ... FROM ... alone is already a complete query. Everything else is optional.

Demo 03

Every question twice: the pandas line you already wrote, then the same question in SQL.

pandas → SQL → match: True

- 01 Build the database: `to_sql`, and a file appears on disk
- 02 Filter: the same 33,317 wrong picks, both ways
- 03 Aggregate + sort: top pickers, the whole anatomy live
- 04 Join: and the warning, plain JOIN is inner
- 05 From query to chart: `read_sql` hands back a DataFrame
- 06 The AI moment: a translator between two tools
- 07 The pickle: one file in the folder is not a table

Same five verbs, two spellings.

VERB	PANDAS	SQL
select	<code>df[["a", "b"]]</code>	SELECT a, b
the table itself	<code>df</code>	FROM df
filter	<code>df[df["x"] == 1]</code>	WHERE x = 1
aggregate	<code>.groupby("a").size()</code>	GROUP BY a + COUNT (*)
sort	<code>.sort_values("n", ascending=False)</code>	ORDER BY n DESC
head	<code>.head(10)</code>	LIMIT 10
join	<code>a.merge(b, on="k", how="inner")</code>	FROM a JOIN b ON a.k = b.k
derive	<code>df["c"] = df["a"] * 2</code>	SELECT a * 2 AS c

Two spellings, one truth: if the numbers differ, one of them is wrong, and it is usually **the join**.

First-contact routine, v2.

FROM LAST WEEK

- ① head
- ② shape
- ③ dtypes
- ④ describe
- ⑤ value_counts + one chart

Five moves that tell you what a table *is*.

⑥

Two questions about the key: is it unique on the reference side (`.is_unique`), and does it mean the same thing on both sides?

⑦

Which columns have holes, and are they structural?
`.isna().mean()`

⑧

Row count before and after **every** join

⑨

assert what must be true, and let the notebook stop you

⑩

Don't trust a function you haven't tested. **Build a five-row toy table where you know the answer, and look.**

Six to nine separate “I merged the tables” from “**I merged the tables and the numbers are right**”.

Try it yourself.

1

Events per event type, in SQL

You already know the pandas answer. That's your answer key.

2

Which sector has the most cells, and is it the one with the most picks?

One join, two groupbys. Layout is not demand.

3

Stretch: average picks per task, by zone

Two aggregations stacked: count first, then average the counts.

Six things to take home.

- 1 Five verbs cover all single-table work, and **a filter is a mask**.
- 2 **groupby = split → apply → combine**, and it leaves out empty keys unless you ask for them.
- 3 A join is VLOOKUP that returns every match and can also delete rows. The default is `inner`.
- 4 A join never errors, so **count rows before and after**. Counting will not catch a wrong key.
- 5 SQL is the same verbs where company data lives. Pull with SQL, polish with pandas.
- 6 **A toy table answers any question about a function faster than the docs.**

OPEN QUESTION

The pay guarantee is per *shift*, and there is no shift column. Some columns don't exist. You define them, then defend the definition.

NEXT WEEK

Statistics and charts that don't lie: means versus medians on skewed data. A second warehouse enters the story, and **HW1 goes out**.